

KERNEL WINDOW PEEPING THROUGH /PROC



This article, which is part of the series on Linux device drivers, demonstrates the creation and usage of files under the `/proc` virtual filesystem.

Part—16

After many months, Shweta and Pugs got together for some peaceful technical romancing. All through, they had been using all kinds of kernel windows, especially through the `/proc` virtual file-system (using `cat`), to help them decode various details of Linux device drivers. Here's a non-exhaustive summary listing:

- `/proc/modules`—dynamically loaded modules
- `/proc/devices`—registered character and block major numbers
- `/proc/iomem`—on-system physical RAM and bus device addresses
- `/proc/ioports`—on-system I/O port addresses (especially for x86 systems)
- `/proc/interrupts`—registered interrupt request numbers
- `/proc/softirqs`—registered soft IRQs
- `/proc/kallsyms`—running kernel symbols, including from loaded modules
- `/proc/partitions`—currently connected block devices and their partitions
- `/proc/filesystems`—currently active filesystem drivers
- `/proc/swaps`—currently active swaps
- `/proc/cpuinfo`—information about the CPU(s) on the system
- `/proc/meminfo`—information about the memory on the system, viz., RAM, swap, ...

Custom kernel windows

"Yes, these have been really helpful in understanding and debugging Linux device drivers. But is it possible for us to also provide some help? Yes, I mean can we create one such kernel window through `/proc`?" asked Shweta.

"Why just one? You can have as many as you want. And it's

simple—just use the right set of APIs, and there you go."

"For you, everything is simple," Shweta grumbled.

"No yaar, this is seriously simple," smiled Pugs. "Just watch me creating one for you," he added.

And in a jiffy, Pugs created the `proc_window.c` file below (download it from http://www.linuxforu.com/article_source_code/mar12/device_driver.zip):

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/proc_fs.h>
#include <linux/jiffies.h>

static struct proc_dir_entry *parent, *file, *link;
static int state = 0;

int time_read(char *page, char **start, off_t off, int count, int
*eof, void *data) {
    int len, val;
    unsigned long act_jiffies;

    len = sprintf(page, "state = %d\n", state);
    act_jiffies = jiffies - INITIAL_JIFFIES;
    val = jiffies_to_msecs(act_jiffies);
    switch (state) {
        case 0:
            len += sprintf(page + len, "time = %ld jiffies\n",
act_jiffies);
            break;
        case 1:
            len += sprintf(page + len, "time = %d msecs\n", val);
```